Final exam in
TDT4125 Algorithm construction, advanced course

Exam date	June 3, 2010
Exam time	0900–1300
Grading date	June 24
Language	English
Contact during exam	Magnus Lie Hetland (ph. 91851949)
Aids	All printed/handwritten; specific, simple calculator

Please read the entire exam before you start, plan your time, and prepare any questions for when the teacher comes to the exam room. Make your assumptions clear where necessary. Keep your answers short and concise. Long explanations that do not directly answer the questions are given little or no weight. All subproblems are given equal weight.

Problem 1

a. Give an example of a directed graph where all nodes can be reached from a start-node *r*, but where a version of Prim's algorithm that takes edge direction into account, started at *r*, will not yield the min-cost arborescence.

b. Show that if the vertex cover problem is in co-NP then NP = co-NP.

Problem 2

a. Describe the general operation of genetic programming.

b. Describe the main difference between genetic programming and other evolutionary algorithms.

c. Describe how to do genetic programming where search is part of the fitness evaluation, and how that differs from normal genetic programming.

In the longest path problem you are to find the longest path in a graph *G* without visiting the same node more than once.

d. Describe a heuristic search algorithm for finding the longest path in a graph with cycles.

Problem 3

a. Explain the concepts of *recall* and *precision* in search, and what they are used for.

b. Create an inverted file index for the following documents:

doc1: "this is the first one"

doc2: "this is not"

doc3: "neither is this"

c. Explain how a search engine can be made to scale linearly both with regard to data volume and query volume.

Problem 4

For a graph $G = (V, E)$ a *matching* is a subset of E where no edges share nodes. A *maximal* matching is a matching that cannot be extended by adding any more edges from E.

a. Explain briefly how the nodes linked to a maximal matching form an approximation with $\rho = 2$ for the vertex cover problem.

Assume that you are given a directed graph $G = (V, E)$. Let the problem A be finding a largest possible (that is, of maximum cardinality) subset of E that yields an acyclic subgraph.

b. Describe an approximation algorithm for A. Give the running time (in θ notation) and approximation ratio, ρ. Explain your reasoning.

Problem 5

PhD student Lurvik is designing a cache for objects of varying size, and wishes to use a Least Recently Used scheme – a Size-dependent LRU (SdLRU). The requirements for the cache structure is being able to quickly access the elements in the cache, and quickly swapping out the contents of the cache with incoming objects.

At a given time in the lifetime of the cache, the cache C is a set of objects $i \in C$. Each object has a size S_i, and the time since the last time the object was used, T_i. The cache has a capacity of K (maximum sum of object sizes). Lurvik is thinking that the value for a cache item is inversely proportional to its age, $1/T_i$. When a new object with size R appears, the cache must evict enough objects to make room for the new one. At the same time, the total value of the evicted objects should be as low as possible.

We introduce a decision variable $q_i \in \{0,1\}$ so that if $q_i=1$, we wish to evict the object from the cache, and if $q_i=0$, we wish to keep it. Thus, what we want is to optimize according to the following criteria:

Minimize $\sum_{i \in C} q_i / T_i$

So that $\sum_{i \in C} q_i S_i \geq R$

We ignore any problems related to cache fragmentation.

a. Describe an algorithm that performs the optimization.

b. Is Lurvik's plan a good one? Explain why this is not a good choice.

c. Your arguments for why this isn't such a good idea are slowly sinking in, and Lurvik suggests an alternative: Remove the object with lowest value/cost, $1/(S_i T_i)$, until there is enough room in the cache. Describe an efficient algorithm for implementing this strategy.

Problem 6

You have a string of n characters $s[1..n]$ where each "character" actually represents a phoneme extracted by analyzing speech and breaking the sound into a sequence of atomic and language specific sound primitives (called

phonemes). In addition, a dictionary operator is available that finds the most likely word associated with any sequence of phonemes. The operator can both return the matching word and the associated error by interpreting the sequence of phonemes as the proposed word:

WORD(w): Returns a string with the English word being the most likely interpretation of the phoneme sequence $w = s[i, i+1, \ldots, j]$

WORD_ERROR(w): Returning a non-negative number representing the error or cost of interpreting the phoneme sequence $w = s[i, i+1, \ldots, j]$ as WORD(w).

Assume that both WORD(w) and WORD_ERROR(w) can be computed in constant time ($O(1)$).

a. Give pseudo-code to describe an algorithm that finds the most likely (that is, the cheapest) sequence of English words that can be extracted from the phoneme sequence $s[1, \ldots, n]$. Mathematically, this means that you need to decompose the sequence of n phoneme characters into a sequence of k words, [WORD(w_1), … , WORD(w_k)] (where k to begin with is unknown), represented by k consecutive intervals in the phoneme string $s[\ldots]$ such that the total error, given by

$$\text{WORD\_ERROR}(w_1) + \ldots + \text{WORD\_ERROR}(w_k)$$

is minimized. What is the running time of your algorithm? (in θ notation)?

b. Your proposed algorithm from **a** might produce individually valid words, but sentences that are non-optimal or not even meaningful for a human reader. Discuss briefly how the algorithm from **a** can be extended with heuristics to produce better sentences (or word sequences).